Gno.land

Whitepaper

Jae Kwon

March 9, 2026

Contents

1 Executive Summary **3**

2 Interrealm Specification **4**
2.1 Introduction . . . 4
2.1.1 Interrealm Programming Context . . . 4
2.1.2 Realm-Context and Realm-Storage-Context . . . 4
2.1.3 Design Goals . . . 5
2.2 Object Model and Realm Storage . . . 7
2.3 Crossing-Functions and Crossing-Methods . . . 8
2.3.1 Definition . . . 9
2.3.2 Realm-Context Rules . . . 10
2.3.3 Implicit Realm-Storage Borrowing . . . 10
2.3.4 Crossing-Method Semantics . . . 11
2.4 Realm Boundaries . . . 12
2.5 Realm-Transaction Finalization . . . 12
2.6 Readonly Taint Specification . . . 12
2.7 `panic()` and `revive(fn)` . . . 14
2.8 `attach()` . . . 15
2.9 `safely(cb func())` . . . 15
2.10 Guidelines . . . 15
2.11 Message Types and Testing . . . 15
2.11.1 MsgCall . . . 15
2.11.2 MsgRun . . . 16
2.11.3 MsgAddPackage . . . 17
2.11.4 Testing overrides with stdlibs/testing . . . 17
2.12 Implementation . . . 19
2.13 Proposed Changes . . . 19

3 Gno Language **20**
3.1 Gno vs Existing Smart Contract Platforms . . . 21
3.2 Interrealm Programming . . . 23

4 Gno.land Blockchain **25**

4.1 Use Case: Open Programmable Knowledge Base 25
4.2 Use Case: Home Computing Chains 28
4.3 Use Case: Supply Chain Transparency and Food Sovereignty 29
4.4 Other Use Cases 29
4.5 Gno.land Constitution 30
4.6 GnoWeb Browser 30

5 Future Work **32**

1 Executive Summary

Gno is the first general-purpose multi-user programming language. It extends Go—a language used by millions of developers—with language-level primitives for multi-user programming: explicit realm boundaries, automatic object persistence, and a shared garbage-collected heap across trust domains.

Every existing programming language assumes a single trusted user. Interactions between users' programs require extra-language frameworks (RPC, serialization, middleware) that add complexity and surface area for exploits. Gno eliminates this: importing and calling another user's code uses the same syntax and type system as importing a library.

Key technical contributions:

- **Interrealm specification.** A formal model for multi-user program interaction with explicit crossing semantics (`cross`/`cur realm`), readonly taints on external objects, and realm-transaction finalization. Every trust boundary is visible in source code and verifiable by the compiler.
- **Automatic object persistence.** Ordinary Go structs persist automatically across transactions with Merkle-ization and garbage collection handled by the runtime. No databases, ORMs, or serialization code required.
- **Shared object graph.** Objects in one realm can reference objects in another realm with language-level type safety. The garbage collector spans trust boundaries. No `externref`, no manual memory management, no custom serialization.
- **Immutable packages as trust anchors.** Pure packages (`/p/`) are immutable on-chain, enabling two mutable realms to trustlessly cooperate through shared immutable code—a guarantee no other smart contract platform provides at the package-system level.

Gno vs alternatives: Solidity's implicit `msg.sender` changes have caused billions in exploits; Move requires a bespoke language with no shared heap; WASM-based systems cannot persist individual objects without serializing entire memory. Gno gives Go developers a familiar language with multi-user safety built in.

Gno.land is a blockchain built on Tendermint2 that runs the GnoVM, serving as the first multi-user language-based operating system. This paper specifies the interrealm model, the Gno language extensions, and the Gno.land blockchain architecture.

2 Interrealm Specification

2.1 Introduction

All modern popular programming languages are designed for a single programmer user. Programming languages support the importing of program libraries natively for components of the single user's program, but this does not hold true for interacting with components of another user's (other) program. Gno is an extension of the Go language for multi-user programming. Gno allows a massive number of programmers to iteratively and interactively develop a single shared program such as Gno.land.

The added dimension of the program domain means the language should be extended to best express the complexities of programming in the inter-realm (inter-user) domain. In other words, Go is a restricted subset of the Gno language in the single-user context. (In this analogy client requests for Go web servers don't count as they run outside the server program).

2.1.1 Interrealm Programming Context

Gno.land supports three types of packages: - **Realms (`/r/`)**: Stateful user applications (smart contracts) that maintain persistent state between transactions - **Pure Packages (`/p/`)**: Stateless libraries that provide reusable functionality - **Ephemeral Packages (`/e/`)**: Temporary code execution with MsgRun which allows a custom main() function to be run instead of a single function call as with MsgExec.

For an overview of the different package types in Gno (`/p/`, `/r/`, and `/e/`), see Anatomy of a Gno Package.

Interrealm programming refers to the ability of one realm to call functions in another realm. This can occur between: - Regular realms (`/r/`) calling other regular realms via MsgExec and MsgRun. - Ephemeral realms (`/e/`) calling regular realms via MsgRun (like main.go)

The key concept is that code executing in one realm context can interact with and call functions in other realms while leveraging the language syntax rules of Go, enabling complex multi-user interactions while maintaining clear boundaries and permissions.

2.1.2 Realm-Context and Realm-Storage-Context

All logic in Gno executes under two contexts that together govern identity and persistence:

Realm-context determines `runtime.CurrentRealm()` and `runtime.PreviousRealm()`. It controls identity and agency: who is the current actor and who called them. The realm-context has an associated Gno address from which native coins can be sent and received. It changes only on explicit cross-calls (`fn(cross, ...)`).

Realm-storage-context determines where new and modified objects are persisted during realm-transaction finalization. It changes on explicit cross-calls *and* on implicit borrow-crosses (calling a non-crossing method on a real object residing in a different realm). The realm-storage-context is not directly accessible at runtime and has no associated address.

After an explicit cross-call, both contexts refer to the same realm. They diverge when calling a non-crossing method of a real object residing in a different realm — the realm-storage-context shifts to the receiver's realm while the realm-context stays the same.

Call type	Realm-ctx?	Storage-ctx?	Boundary?	Finalizes?
`fn(cross,...)` to same realm	Yes*	No	Yes	Yes
`fn(cross,...)` to diff. realm	Yes	Yes	Yes	Yes
`fn(nil,...)` (non-crossing)	No	No	No	No
Non-crossing method, real recv. in same realm	No	No	No	No
Non-crossing method, real recv. in diff. realm	No	Yes	Yes	Yes
Non-crossing method, unreal recv.	No	No	No	No
Non-crossing function	No	No	No	No

* `runtime.CurrentRealm()` returns the same realm, but `runtime.PreviousRealm()` shifts — what was current becomes previous. See Realm Boundaries for definitions of boundary and finalization.

2.1.3 Design Goals

Caveat: The interrealm specification does not secure applications against arbitrary code execution. It is important for realm logic (and even p package logic) to ensure that arbitrary (variable) functions (and similarly arbitrary interface methods) are not provided by malicious callers; such arbitrary functions and methods whether crossing (or non-crossing) will inherit the previous realm (or both current and previous realms) and could abuse these realm-contexts. It does not make sense for any realm user to cross-call an arbitrary function or method as it loses agency while being marked as the responsible caller by the callee's runtime previous realm. This problem is worse when calling a non-crossing function or method. It can be reasonable when such variable functions or interface values are restricted in other ways such as by whitelisting by a DAO upon careful inspection of every such variable function or interface value (both its type declaration as well as its state).

P package code should behave the same even when copied verbatim in a realm package; and likewise non-crossing code should behave the same when copied verbatim from one realm to another. Otherwise there will be lots of security related bugs from user error.

Realm crossing with respect to `runtime.CurrentRealm()` and `runtime.PreviousRealm()` must be explicit and warrants type-checking; because a crossing-function of a realm should be able to call another crossing-function of the same realm without necessarily crossing (changing the realm-context). Sometimes the previous realm and current realm must be the same realm, such as when a realm consumes a service that it offers to external realms and users.

For Go developers familiar with blockchain VMs like the EVM: in Solidity, calling another contract implicitly shifts `msg.sender`, making it easy to introduce reentrancy bugs or misattribute the caller. Gno's `cross` keyword makes every realm-context transition visible in source code and verifiable by the compiler, eliminating this class of bugs by construction.

Where a real object resides should not matter too much, as it is often difficult to predict. Thus the realm-context as returned by `runtime.PreviousRealm()` and `runtime.CurrentRealm()` should not change with non-crossing method calls, and the realm-storage-context should be determined for non-crossing methods only by the realm-storage of the receiver. The realm-storage of a receiver should only matter for when elements reside in external realm-storage and direct dot-selector or index-expression access of sub-elements are desired of the aforementioned element.

A method should be able to modify the receiver and associated objects of the same realm-storage as that of the receiver.

A method should be able to create new objects that reside in the same realm by association in order to maintain storage realm consistency and encapsulation and reduce fragmentation.

It is difficult to migrate an object from one realm to another even when its ref-count is 1; such an object may be deep with many descendants of ref-count 1 and so performance is unpredictable.

Code declared in p packages (or declared in "immutable" realm packages) can help different realms enforce contracts trustlessly, even those that involve the caller's current realm. Otherwise two mutable (upgradeable) realms cannot export trust unto the chain because functions declared in those two realms can be upgraded. This is analogous to Go interfaces but stronger: a Go interface guarantees method signatures, while a Gno p-package guarantees *behavior* — the implementation is immutable on-chain, so both parties can trust the logic without trusting each other.

Both `fn(cross, ...)` and `func fn(cur realm, ...){...}` may become special syntax in future Gno versions.

```go
// realm /r/alice/alice
package alice

var object any

func SetObject(cur realm, obj any) {
    object = obj
}
```

```go
// package /p/bob/types
package types

type UserProfile struct {
    Name string
    ...
}
```

```go
// realm /r/bob/bob
package bob
```

```go

import "gno.land/r/alice/alice" // import external realm package
import "gno.land/p/bob/types"   // import external library package

func Register(cur realm, name string) {
    prof := types.UserProfile{Name: name}
    alice.SetObject(cross, prof)
}
```

The Gno language is extended to support a `context.Context`-like argument to denote the current realm-context of a Gno function. This allows a user realm function to call itself safely as if it were being called by an external user, and helps avoid a class of security issues that would otherwise exist.

```go
// realm /r/alice/mail

func SendMail(cur realm, text string) {
    if text == "" {
        // runtime.PreviousRealm() is preserved for recursive call.
        SendMail(nil, "<empty>")
    }
    caller := runtime.PreviousRealm()
    if inBlacklist(caller) {
        // runtime.PreviousRealm() becomes self; message from self to self.
        SendMail(cross, fmt.Sprintf("blacklisted caller %v blocked", caller))
    } else {
        // sendMailPrivate not exposed to external callers.
        sendMailPrivate(text)
    }
}
```

2.2 Object Model and Realm Storage

Unlike other blockchain platforms where developers must manually serialize state into key-value stores, Gno persists ordinary Go structs and slices automatically. A Go developer can write familiar data structures — trees, linked lists, maps of structs — and they survive across transactions with no explicit marshaling. The runtime handles Merkleization and garbage collection transparently.

Every object in Gno is persisted on disk with additional metadata including the object ID and an optional OwnerID (if persisted with a ref-count of exactly 1). The object ID is only set at the end of a realm-transaction during realm-transaction finalization (more on that later). A GnoVM transaction is composed of one or many scoped (stacked) realm-transactions.

```go
type ObjectInfo struct {
    ID       ObjectID  // set if real.
    Hash     ValueHash `json:",omitempty"` // zero if dirty.
    OwnerID  ObjectID  `json:",omitempty"` // parent in the ownership tree.
    ModTime  uint64    // time last updated.
    RefCount int       // for persistence. deleted/gc'd if 0.

    // Object has multiple references (refcount > 1) and is persisted separately
    IsEscaped bool `json:",omitempty"` // hash in iavl.
    ...
}
```

When an object is persisted during realm-transaction finalization the object becomes "real" (as in it is really persisted in the virtual machine state) and is said to "reside" in the realm; and otherwise is considered "unreal". New objects instantiated during a transaction are always unreal; and during finalization such objects are either discarded (transaction-level garbage collected) or become persisted and real.

Unreal (new) objects that become referenced by a real (persisted) object at runtime will get their OwnerID set to the parent object's storage realm, but will not yet have its object ID set before realm-transaction finalization. Subsequent references at runtime of such an unreal object by real objects residing in other realms do not override the OwnerID initially set, so during realm-transaction finalization it ends up residing in the first realm it became associated with (referenced from). Unreal objects that become persisted but were never directly referenced by any real object during runtime will only get their OwnerID set to the realm of the first real ancestor.

Real objects with ref-count of 1 have their hash included in the sole parent object's serialized byte form, thus an object tree of only ref-count 1 descendants are Merkle-hashed completely.

When a real or unreal object ends up with a ref-count of 2 or greater during realm-transaction finalization its OwnerID is set to zero and the object is considered to have "escaped". When such a real object is persisted with ref-count of 2 or greater it is forever considered escaped even if its ref-count in later transactions is reduced to 1. Escaped real objects do not have their hash included in the parent objects' serialized byte form but instead are Merkle-ized separately in an iavl tree of escaped object hashes (keyed by the escaped object's ID) for each realm package. (This is implemented as a stub but not yet implemented for the initial release of Gno.land.)

A real object can only be directly mutated through dot-selectors and index-expressions if the object resides in the same realm as the current realm-storage-context. Unreal objects can always be directly mutated if their elements are directly exposed.

Exposed values accessed through dot-selectors and index-expressions from external realm logic are tainted read-only. For the full rules see Readonly Taint Specification.

2.3 Crossing-Functions and Crossing-Methods

Realm crossing occurs when a crossing function (declared as `func fn(cur realm, ...){...}`) is called with the Gno `fn(cross, ...)` syntax.

```
package main
import "gno.land/r/alice/realm1"

func main() {
    bread := realm1.MakeBread(cross, "flour", "water")
}
```

(In Linux/Unix operating systems user processes can cross-call into the kernel by calling special syscall functions, but user processes cannot directly cross-call into other users' processes. This makes the GnoVM a more complete multi-user operating system than traditional operating systems.)

Besides explicit realm crossing via the `fn(cross, ...)` Gno syntax, implicit realm crossing occurs when calling a method of a receiver object stored in an external realm. Implicitly crossing into (borrowing) a receiver object's storage realm allows the method to directly modify the receiver as well as all other objects directly reachable from the receiver stored in the same realm as the receiver. Unlike explicit crosses, implicit crosses do not shift or otherwise affect the current realm context; `runtime.CurrentRealm()` does not change unless a method is called like `receiver.Method(cross, args...)`.

Realms hold objects in residence and they also have a Gno address to send and receive coins from. Coins can only be spent from the current realm context.

2.3.1 Definition

A crossing-function or crossing-method is that which is declared in a realm and has as its first argument `cur realm`. The `cur realm` argument must appear as the first argument of a crossing-function or crossing-method's argument parameter list. To prevent confusion it is illegal to use anywhere else, and cannot be used in p packages.

The current realm-context and realm-storage-context changes when a crossing-function or crossing-method is called with the `cross` keyword in the first argument as in `fn(cross, ...)`. Such a call is called a "cross-call" or "crossing-call".

```
package main
import "gno.land/r/alice/extrealm"

func MyMakeBread(cur realm, ingredients ...any) { ... }

func main(cur realm) {
    MyMakeBread(cross, "flour", "water") // ok -- cross into self.
    extrealm.MakeBread(cross, "flour", "water") // ok -- cross into extrealm
}
```

When a crossing-function or crossing-method is called with `nil` as the first argument instead of `cross` it is called a non-crossing-call; and no realm-context nor realm-storage-context change takes place.

```
package main
import "gno.land/r/alice/extrealm"

func MyMakeBread(cur realm, ingredients ...any) { ... }

func main(cur realm) {
    MyMakeBread(nil, "flour", "water") // ok -- non-crossing.
    extrealm.MakeBread(nil, "flour", "water") // invalid -- external realm
    function
}
```

To prevent confusion a non-crossing-call of a crossing-function or crossing-method declared in a realm different than that of the caller's realm-context and realm-storage-context will result in either a type-check error; or a runtime error if the crossing-function or crossing-method is variable.

2.3.2 Realm-Context Rules

All functions in Gno execute under a realm context as determined by the call stack. Objects that reside in a realm can only be modified if the realm context matches.

A function declared in p packages when called:

- inherits the last realm for package declared functions and closures.
- inherits the last realm when a method is called on unreal receiver.
- implicitly crosses to the receiver's resident realm when a method of the receiver is called. The receiver's realm is also called the "borrow realm".

A function declared in a realm package when called:

- explicitly crosses to the realm in which the function is declared if the function is declared as `func fn(cur realm, ...){...}` (with `cur realm` as the first argument). The new realm is called the "current realm".
- otherwise follows the same rules as for p packages.

`runtime.CurrentRealm()` returns the current realm-context that was last cross-called to. `runtime.PreviousRealm()` returns the realm-context cross-called to before the last cross-call. All cross-calls are explicit with the `cross` keyword, as well as non-crossing-calls of crossing-functions and crossing-methods with `nil` instead of `cross`.

A crossing function declared in the same realm package as the callee may be called like `fn(cross, ...)` or `fn(cur, ...)`. When called with `fn(cur, ...)` there will be no realm crossing, but when called like `fn(cross, ...)` there is technically a realm crossing and the current realm and previous realm returned are the same.

The current realm and previous realm do not depend on any implicit crossing to the receiver's borrowed/storage realm even if the borrowed realm is the last realm of the call stack. In other words `runtime.CurrentRealm()` may differ from the Machine's active storage realm (internally `m.Realm`, i.e. the borrow realm) when a method is called on a receiver residing in a foreign realm.

2.3.3 Implicit Realm-Storage Borrowing

Besides (explicit) realm-context changes via the `fn(cross, ...)` cross-call syntax, implicit realm-storage-context changes occur when calling a non-crossing method of a receiver object residing in different realm-storage. The realm-storage-context "borrows" to the receiver's realm, which allows the method to directly modify its receiver and any objects reachable from it that reside in the same realm-storage. The realm-context does not change (so `runtime.CurrentRealm()` and `runtime.PreviousRealm()` are unaffected; the agency of the caller remains the same). However, objects reachable from the receiver but residing in a *different* realm-storage cannot be modified — the `DidUpdate()` guard in the runtime enforces that only objects belonging to the borrowed realm can be mutated.

This borrowing behavior allows non-crossing methods of receivers to behave the same whether declared in a realm package or p package: p package code copied over to a realm package, or realm package code copied over to another realm, have the exact same behavior. Crossing methods of a realm package would still behave differently when copied over to another realm, as crossing-methods always change the realm-context and realm-storage-context to the declared realm.

2.3.4 Crossing-Method Semantics

When a method is a crossing-method called as `receiver.Method(cross, args...)`, both the realm-context and realm-storage-context change to that of the realm package in which the type/method is declared (which is not necessarily the same as where the receiver resides). Such a crossing method-call cannot directly modify the real receiver if it happens to reside in an external realm that differs from where the type and methods are declared; but it can modify any unreal receiver or unreal reachable objects. As mentioned previously a non-crossing-call of a crossing-method will fail during type-checking or at runtime if the receiver resides in an external realm-storage.

Calls of methods on receivers residing in realms different from the current realm must *not* be called like `fn(cross, ...)` if the method is not a crossing function itself, and vice versa. Or it could be said that implicit crossing is not real realm crossing. (When you sign a document with someone else's pen it is still your signature; signature:pen :: current:borrowed)

A crossing method declared in a realm cannot modify the receiver if the object resides in a different realm. However not all methods are required to be crossing methods, and crossing methods may still read the state of the receiver (and in general anything reachable is readable).

```go
// Type and methods declared in /r/alice/tokens
package tokens

type Token struct {
    Owner   string
    Balance int
}

// Non-crossing method: borrows receiver's storage realm.
func (t *Token) Debit(amount int) {
    t.Balance -= amount
}

// Crossing method: realm-context and realm-storage-context
// shift to /r/alice/tokens (where the type is declared).
func (t *Token) Transfer(cur realm, to string, amount int) {
    t.Balance -= amount // FAILS if receiver resides outside /r/alice/tokens
}
```

```go
// /r/bob/bob stores a Token
package bob

import "gno.land/r/alice/tokens"

var myToken *tokens.Token // persisted in /r/bob/bob

func Spend(_ realm) {
    // Non-crossing call: storage-context borrows to /r/bob/bob
    // (receiver's realm). Debit() CAN modify myToken.
    myToken.Debit(10) // ok: borrowed storage matches receiver

    // Crossing call: storage-context shifts to /r/alice/tokens
    // (where the type is declared), not /r/bob/bob (where myToken resides).
    // Transfer() CANNOT directly modify myToken.
    myToken.Transfer(cross, "g1...", 10) // fails: receiver in external realm
}
```

New unreal objects reachable from the borrowed realm (or current realm if there was no method call that borrowed) become persisted in the borrowed realm (or current realm) upon finalization of the foreign object's method (or function). (When you put an unlabeled photo in someone else's scrapbook the photo now belongs to the other person.) In the future the `attach()` function will prevent a new unreal object from being taken.

For how crossing rules apply to MsgCall, MsgRun, and package initialization, see Message Types and Testing.

2.4 Realm Boundaries

A realm boundary is defined as a change in realm in the call frame stack from one realm to another, whether explicitly crossed with `fn(cross, ...)` or implicitly borrow-crossed into a different receiver's storage realm. A realm may cross into itself with an explicit cross-call.

When a crossing-function or crossing-method is cross-called it shifts the "current" runtime realm-context to the "previous" runtime realm-context such that `runtime.PreviousRealm()` returns what used to be returned with `runtime.CurrentRealm()` before the realm boundary. The current realm-storage-context is always set to that of realm-context after cross-calling.

For which call types create boundaries, see the summary table above. Every crossing-call creates a realm boundary even when there is no resulting change in realm-context or realm-storage-context. A non-crossing-call of a crossing-function or crossing-method (`fn(nil, ...)`) never creates a realm boundary.

2.5 Realm-Transaction Finalization

Realm-transaction finalization occurs when returning from a realm boundary. When returning from a cross-call (with `cross`) realm-transaction finalization will occur even with no change of realm-context or realm-storage-context. Realm-transaction finalization does NOT occur when returning from a non-crossing-call of a method of an unreal receiver or a real receiver that resides in the same realm-storage-context as that of the caller.

During realm-transaction finalization all new reachable objects are assigned object IDs and stored in the current realm-storage-context; and ref-count-zero objects deleted (full "disk-persistent cycle GC" will come after launch); and any modified ref-counts and new Merkle hash root computed.

2.6 Readonly Taint Specification

Go's language rules for value access through dot-selectors & index-expressions are the same within the same realm, but exposed values through dot-selector & index-expressions are tainted read-only when performed by an external realm.

The readonly taint prevents the direct modification of real objects by any logic, even from logic declared in the same realm as that of the object's storage-realm.

A realm cannot directly modify another realm's objects without calling a function that gives permission for the modification to occur.

For example `externalrealm.Foo` is a dot-selector expression on an external object (package) so the value is tainted with the `N_Readonly` attribute.

The same is true for `externalobject.FieldA` where `externalobject` resides in an external realm.

The same is true for `externalobject[0]`: direct index expressions also taint the resulting value with the `N_Readonly` attribute.

The same is true for `externalobject.FieldA.FieldB[0]`: the readonly taint persists for any subsequent direct access, so even if FieldA or FieldB resided in the caller's own realm-context or realm-storage the result is tainted readonly.

A Gno package's global variables even when exposed (e.g. `package realm1; var MyGlobal int = 1`) are safe from external manipulation (e.g. `import "xxx/realm1"; realm1.MyGlobal = 2`) by the readonly taint when accessed directly by dot-selector or index-expression from external realm logic; and also by a separate `DidUpdate()` guard when accessed by other means such as by return value of a function and the return value is real and external.

A function or method's arguments and return values retain and pass through any readonly taint from caller to callee. Even if a realm's function (or method) returns an untainted real object, the runtime guard in `DidUpdate()` prevents it from being modified by an external realm-storage-context.

For a realm (user) to manipulate an untainted object residing in an external realm, a function (or method) can be declared in the external realm which references and modifies the aforementioned untainted object directly (by a name declared outside of the scope of said function or method). Or, the function can take in as argument an untainted real object returned by another function.

Besides protecting against writing by direct access, the readonly taint also helps prevent a class of security issue where a realm may be tricked into modifying something that it otherwise would not want to modify. Since the readonly taint prohibits mutations even from logic declared in the same realm, it protects realms against mutating its own object that it doesn't intend to: such as when a realm's real object is passed as an argument to a mutator function where the object happens to match the type of the argument.

Objects returned from functions or methods are not readonly tainted. So if `func (eo object) GetA() any { return eo.FieldA }` then `externalobject.GetA()` returns an object that is not tainted assuming eo.FieldA was not otherwise tainted. While the parent object `eo` is still protected from direct modification by external realm logic, the returned object from `GetA()` can be passed as an argument to logic declared in the residing realm of `eo.FieldA` for direct mutation.

Whether or not an object is readonly tainted it can always be mutated by a method declared on the receiver.

```
// /r/alice

var blacklist []string

func GetBlacklist() []string {
    return blacklist
}

```

```go
func FilterList(cur realm, testlist []string) { // blanks out blacklist items from testlist
    for i, item := range testlist {
        if contains(blacklist, item) {
            testlist[i] = ""
        }
    }
}
```

This is a toy example, but you can see that the intent of `FilterList()` is to modify an externally provided slice; yet if you call `alice.FilterList(cross, alice.GetBlacklist())` you can trick alice into modifying its own blacklist–the result is that alice.BlackList becomes full of blank values.

With the readonly taint `var Blacklist []string` solves the problem for you; that is, /r/bob cannot successfully call `alice.FilterList(cross, alice.Blacklist)` because `alice.Blacklist` is readonly tainted for bob.

The problem remains if alice implements `func GetBlacklist() []string { return Blacklist }` since then /r/bob can call `alice.FilterList(cross, alice.GetBlacklist())` and the argument is not readonly tainted.

Future versions of Gno may also expose a new modifier keyword `readonly` to allow for return values of functions to be tainted as readonly. Then with `func GetBlacklist() readonly []string` the return value would be readonly tainted for both bob and alice.

2.7 panic() and revive(fn)

`panic()` behaves the same within the same realm boundary, but when a panic crosses a realm boundary (as defined in Realm Boundaries) the Machine aborts the program. This is because in a multi-user environment it isn't safe to let the caller recover from realm panics that often leave the state in an invalid state.

This would be sufficient, but we also want to write our tests to be able to detect such aborts and make assertions. For this reason Gno provides the `revive(fn)` builtin.

```go
abort := revive(func() {
    fn := func(_ realm) {
        panic("cross-realm panic")
    }
    fn(cross)
})
abort == "cross-realm panic"
```

`revive(fn)` will execute 'fn' and return the exception that crossed a realm boundary during finalization.

This is only enabled in testing mode (for now), behavior is only partially implemented. In the future `revive(fn)` will be available for non-testing code, and the behavior will change such that `fn()` is run in transactional (cache-wrapped) memory context and any mutations discarded if and only if there was an abort.

TL;DR: `revive(fn)` is Gno's builtin for STM (software transactional memory).

2.8 `attach()`

In future releases of Gno the `attach()` function can be used to associate unreal objects to the current realm-storage-context before being passed into a function declared in an external realm package, or into a method of a real receiver residing in an external realm-context.

2.9 `safely(cb func())`

In future releases of Gno the `safely(cb func())` function may be used to clear the current and previous realm-context as well as any realm-storage-context such that no matter what `cb func()` does the caller does not yield agency to the callee.

For now this can be simulated by implementing an (immutable non-upgradeable) realm crossing-function that cross-calls into itself once more before calling the callback function.

2.10 Guidelines

P package code cannot contain crossing functions. P package code also cannot import R realm packages. But code can call named crossing functions e.g. those passed in as parameters.

You must declare a public realm function to be a crossing function if it is intended to be called by end users, because users cannot MsgCall non-crossing functions (for safety/consistency) or p package functions (there's no point).

Utility functions that are a common sequence of non-crossing logic can be offered in realm packages as non-crossing functions. These can also import and use other realm utility non-crossing functions; whereas p packages cannot import realm packages at all. And convenience/utility functions that are being staged before publishing as permanent p code should also reside in upgradeable realms.

Generally you want your methods to be non-crossing. Because they should work for everyone. They are functions that are pre-bound to an object, and that object is like a quasi-realm in itself, that could possibly reside and migrate to other realms. This is consistent with any p code copied over to r realms; none of those methods would be crossing, and behavior would be the same; stored in any realm, mostly non-crossing methods that anyone can call. Why is a quasi-realm self-encapsulated object in need to modify the realm in which it is declared, by crossing? That's intrusive, but sometimes desired.

You can always cross-call a method from a non-crossing method if you need it.

2.11 Message Types and Testing

2.11.1 MsgCall

MsgCall may only call crossing functions. This is to prevent potential confusion for non-sophisticated users. Non-crossing calls of non-crossing functions of other realms is still possible with MsgRun.

```
// PKGPATH: gno.land/r/test/test
```

```go

func Public(_ realm) {

    // Returns (
    //     addr:<origin_caller>,
    //     pkgpath:""
    // ) == testing.NewUserRealm(origin_caller)
    runtime.PreviousRealm()

    // Returns (
    //     addr:chain.PackageAddress("gno.land/r/test/test"),
    //     pkgpath:"gno.land/r/test/test"
    // ) == testing.NewCodeRealm("gno.land/r/test/test")
    runtime.CurrentRealm()

    // Call a crossing function of same realm with crossing
    AnotherPublic(cross)

    // Call a crossing function of same realm without crossing
    AnotherPublic(cur)
}

func AnotherPublic(_ realm) {
    ...
}
```

2.11.2 MsgRun

```go
// PKGPATH: gno.land/e/g1user/run

import "gno.land/r/realmA"

func main() {
    // Before main() is called there is an implicit
    // crossing from UserRealm(g1user) to
    // CodeRealm(gno.land/e/g1user/run).

    // Returns (
    //     addr:g1user,
    //     pkgpath:""
    // ) == testing.NewUserRealm(g1user)
    runtime.PreviousRealm()

    // Returns (
    //     addr:g1user,
    //     pkgpath:"gno.land/e/g1user/run"
    // ) == testing.NewCodeRealm("gno.land/e/g1user/run")
    runtime.CurrentRealm()

    realmA.PublicNoncrossing()
    realmA.PublicCrossing(cross)
}
```

Realm addresses are derived from package paths via `chain.PackageAddress()` (defined in `gnovm/stdlibs/chain/address.gno`):

- `chain.PackageAddress("gno.land/r/name123/realm")` — bech32 from hash(path)
- `chain.PackageAddress("gno.land/e/g1user/run")` — bech32 substring "g1user"

Therefore in the MsgRun file's `init()` function the previous realm and current realm have different pkgpaths (the origin caller always has empty pkgpath) but the address is the

same.

2.11.3 MsgAddPackage

A realm package's initialization (including `init()` calls) executes with current realm-context of itself. `runtime.PreviousRealm()` refers to the package deployer both in global var decls and inside `init()` functions. After that the package deployer is no longer provided, so packages need to remember the deployer in the initialization phase if needed.

```
// PKGPATH: gno.land/r/test/test

func init() {
    // Returns (
    //     addr:<origin_deployer>,
    //     pkgpath:""
    // ) == testing.NewUserRealm(origin_deployer)
    // Inside init() and global var decls
    // are the only time runtime.PreviousRealm()
    // returns the deployer of the package.
    // Save it here or lose it forever.
    runtime.PreviousRealm()

    // Returns (
    //     addr:chain.PackageAddress("gno.land/r/test/test"),
    //     pkgpath:"gno.land/r/test/test"
    // ) == testing.NewCodeRealm("gno.land/r/test/test")
    runtime.CurrentRealm()
}

// Same as in init().
var _ = runtime.PreviousRealm()
```

```
// PKGPATH: gno.land/e/g1user/run

func init() {
    // Returns (
    //     addr:g1user,
    //     pkgpath:""
    // ) == testing.NewUserRealm(g1user)
    runtime.PreviousRealm()

    // Returns (
    //     addr:g1user,
    //     pkgpath:"gno.land/e/g1user/run"
    // ) == testing.NewCodeRealm("gno.land/e/g1user/run")
    runtime.CurrentRealm()
}
```

The same applies for pure package (`/p/`) initialization. During initialization and tests, `runtime.CurrentRealm()` can return a package path that starts with "/p/". This is because the package is technically still mutable during its initialization phase. After initialization, pure packages become immutable and cannot maintain state.

2.11.4 Testing overrides with stdlibs/testing

The `gnovm/tests/stdlibs/testing/context_testing.gno` file provides functions for overriding frame details from Gno test code.

`testing.SetRealm(testing.NewUserRealm("g1user"))` is identical to `testing.SetOriginCaller`

("g1user"). Both will override the Gno frame to make it appear as if the current frame is the end user signing with a hardware signer. Both will also set `ExecContext.OriginCaller` to that user. One of these will become deprecated.

Gno test cases with `_test.gno` like `TestFoo(t *testing.T)`

```go
// PKGPATH: gno.land/r/user/myrealm
package myrealm

import (
    "chain/runtime"
    "testing"
)

func TestFoo(t *testing.T) {
    // At first OriginCaller is not set.

    // Override the OriginCaller.
    testing.SetRealm(testing.NewUserRealm("g1user"))

    // Identical behavior:
    testing.SetOriginCaller("g1user")

    // This panics now: seeking beyond the overridden origin frame:
    // runtime.PreviousRealm()

    // Simulate g1user cross-calling Public().
    // Produce a new frame to override
    func() {
        testing.SetRealm(testing.NewCodeRealm("gno.land/r/user/myrealm"))

        runtime.PreviousRealm() // "g1user", ""
        runtime.CurrentRealm()  // bech32(hash("gno.land/r/user/myrealm")), "gno.land/r/user/myrealm"

        Public(...) // already in "gno.land/r/user/myrealm"
    }()

    // The following is identical to the above,
    // but not possible in p packages which
    // cannot import realms.
    Public(cross, ...)
}
```

Gno filetest cases with `_filetest.gno`

```go
// PKGPATH: gno.land/r/test/test
package test

import (
    "chain/runtime"
    "testing"

    "gno.land/r/user/myrealm"
)

func init() {
    // XXX Frame not found, there is no deployer for filetests.
    runtime.PreviousRealm()

    // Returns (
    //     addr:chain.PackageAddress("gno.land/r/test/test")
    //     pkgpath:"gno.land/r/test/test"
    // ) == testing.NewCodeRealm("gno.land/r/test/test")
    runtime.CurrentRealm()
```

```
}

func main() {
    // There is assumed to be in "frame -1"
    // a crossing from UserRealm(g1user) to
    // CodeRealm("gno.land/r/test/test") before
    // main() is called, so crossing() here
    // is redundant.

    // Returns (
    //     addr:g1user,
    //     pkgpath:""
    // ) == testing.NewUserRealm(g1user)
    runtime.PreviousRealm()

    // Returns (
    //     addr:g1user,
    //     pkgpath:"gno.land/r/test/test"
    // ) == testing.NewCodeRealm("gno.land/r/test/test")
    runtime.CurrentRealm()

    // gno.land/r/test/test cross-calling
    // gno.land/r/user/myrealm:
    myrealm.Public(cross, ...)
}

// Output:
// XXX
```

2.12 Implementation

Implementation for `runtime.CurrentRealm()` and `runtime.PreviousRealm()` are defined in `gnovm/stdlibs/chain/runtime/native.gno` and related files in the directory, while overrides for testing are defined in `gnovm/tests/stdlibs/testing/context_testing.gno`. All stdlibs functions are available unless overridden by the latter.

2.13 Proposed Changes

`testing.SetOriginCaller()` may be deprecated in favor of `testing.SetRealm(testing.NewOriginRealm(user))`.

`testing.NewCodeRealm(path)` may be renamed to `testing.NewPackageRealm(path)`.

3 Gno Language

We could not have arrived at the final design of the Gno language without the help of the many developers who gave much time of their lives to contribute to this project and the design of the language; specifically the NewTendermint Gno team and Gno community, and especially Manfred, Morgan, Maxwell, Milos, Guilhem, Ray, and Omar.

Gno is the first general purpose *multi-user* programming language.

By "multi-user" we mean a language where (1) multiple users' code and data coexist in a shared persistent heap, (2) trust boundaries between users are enforced by the language itself–not by the operating system, a hypervisor, or ad-hoc middleware–and (3) inter-user interoperability uses the same syntax and type system as intra-user code. Prior systems have achieved subsets of these properties: MUMPS (1966) shares persistent global state across users but has no language-level trust boundaries; MOO (1990) and LPC (1989) provide multi-user object ownership in virtual worlds but are domain-specific MUD languages, not general purpose; Erlang (1986) excels at concurrency but all processes are equally trusted with no per-user security model; Solidity (2015) and Move (2019) handle multi-user smart contracts but lack transparent inter-module interoperability with a shared heap (as detailed below). Gno is the first language to combine all three properties in a general purpose language based on an existing mainstream language (Go).

Gno is a minimal extension of the Go language for multi-user programming. Gno allows a massive number of programmers to iteratively and interactively develop a single shared program such as Gno.land (making it ideal for interoperable smart contract programming). In other words, Go is a restricted subset of the Gno language in the single-user context.

All of our programming languages to date are designed for a single programmer user. All programming languages make the same assumption that there is only one user – the programmer, or program executor user. Whether written in C, C++, Python, Java, Javascript, or Go, it is assumed that all of the dependencies of the program are trusted. If there is a vulnerability in any of the dependencies there is a vulnerability in the program; it is the job of the programmer or program/product manager to ensure that the overall program is free of exploits.

When interacting with programs owned by another user (or process) various techniques are used such as via IPC APIs often generated by tools like Protobuf/GRPC; but such tools add extra complexity, additional surface area for exploits, additional compute complexity, and do not benefit directly from the language's native rules and type-checker–especially for inter-process passing of in-memory object references. In Gno, interacting with another realm's function is just like interacting with any dependency (import and call).

Automatic persistence of memory objects makes databases, ORMs, and serialization generally unnecessary. GnoVM at the end of realm transaction boundaries will "finalize" by scanning all new, modified, and deleted objects and will automatically persist what needs to be saved along with meta information like the Merkle hash, ref count, associated realm and unique number of each object. By making inter-user interoperability native to the language and also by removing the distinction between volatile and persistent memory, Gno eliminates much of the accidental complexity that plagues traditional multi-service

and smart-contract development.

The GnoVM is implemented purely in Go and is a Gno AST interpreter. It is implemented to serve as a reference implementation suitable for production and tinkering. The GnoVM opcodes are expected to change over time to balance simplicity, efficiency, and transpilation targets–perhaps even to hardware. Just as Ethereum was originally implemented in Python we expect future software to be optimized, whether implemented in other languages, or possibly transpiled from the existing GnoVM implementation.

For the latest Gno specs including the inter-realm spec refer to the Gno docs/resources directory.

3.1 Gno vs Existing Smart Contract Platforms

Smart contract platforms like Ethereum allow for many users to upload their application and call other user application logic functions, but Solidity is not a general purpose programming language and has severe limitations that make it suboptimal for solving the task at hand.

First, **Solidity and other existing smart-contract languages/platforms do not support transparent inter-module (inter-user) interoperability with the same language rules as for intra-module dependencies.** That is, an application developer for a smart-contract cannot simply import and call/use another user application's functions and types as if they were library dependencies of the same application. Generally interoperability between different modules are implemented with extra-language frameworks and libraries on top of an incomplete or primitive message-passing agent architecture; such interop function calls generally do not share the same call-stack nor memory space. Solidity's implicit `msg.sender` changes on external calls have caused billions of dollars in exploits (reentrancy, confused-deputy attacks). In Gno the programmer writes `cross` at each call site that changes the realm-context, making every trust boundary visible in the source code and checkable by the compiler–a small syntactic cost for a huge safety gain. Move (used by Sui and Aptos) improves on Solidity with a linear type system that prevents reentrancy, but at the cost of a bespoke language unfamiliar to most developers and without a shared persistent object graph; inter-module calls in Move still pass values by copy or move rather than by shared reference.

Second, **Solidity and other existing smart-contract languages/platforms do not support the automatic persistence and Merkle-ization of in-memory (heap) objects and often require custom serialization logic.** Solidity does not support such a heap at all as all memory for variables are predeclared in the function and as such is not object-oriented and does not have a garbage collector or similar memory-management primitives. WASM-based smart contract systems do not support automatic persistence of objects without persisting the entire memory state of the module. This requires a specialized virtual machine such as the Gno VM which keeps track of every object created, modified, and deleted. Solidity developers manage storage slots and mappings manually; Move developers serialize structs into global storage with explicit `move_to` and `borrow_global` calls. Gno developers write ordinary Go structs and they persist automatically with Merkle-ization–a major reduction in complexity for the common case.

The automatic persistence of in-memory objects of the GnoVM is like a memristor simulator. The advent of AI has created a new market for memristor-based memory systems

where the distinction between RAM volatile memory and persistent disk storage is removed. Urbit is similar but is not based on any general purpose programming language. With memristor-based memory the GnoVM can be further simplified and the performance of applications can be vastly improved without any changes to the Gno language specification.

Third, **Solidity and other existing smart-contract languages/platforms do not support a shared heap memory space for objects to be referenced by external-user objects in a uniform manner by language rules**. Alice cannot simply declare a structure object that references the structure object persisted in Bob's application and trust the garbage collector to retain Bob's object for as long as Alice's object is retained.

Fourth, **the split between immutable p-packages (`gno.land/p/`) and mutable r-realms (`gno.land/r/`) solves a real trust problem that no other smart contract platform addresses at the package-system level.** Two mutable realms cannot trustlessly cooperate because either could upgrade its code, but an immutable p-package can enforce a contract between them. In Solidity, library contracts can be proxied and upgraded, undermining any guarantees they were supposed to provide. In Move, modules are also upgradeable by default. Gno bakes the immutability guarantee into the package path convention itself.

The above differentiating factors of the Gno language allows for the most succinct expression of a single-user application or multi-user application composed of independent modules without the extra complexity from extra-language interop type-checking syntax or frameworks nor of the extra complexity from any database, ORM, or serialization logic.

Shared garbage-collection in a shared (multi-user) graph of object references makes it possible for one's object representing (say) a propositional statement or idea to be easily referenced by an alternative statement or idea, or even be extended by reference with additional commentary, metadata, or even a subreddit-like tree of discussions. Without a shared garbage collector the task of ensuring that references still hold over time without becoming dangling pointers is left up to each inter-application interface at best, requiring custom logic just to handle garbage collection. WebAssembly (WASM) externref support in Go has limitations, particularly in how it handles external memory references. Currently, Rust and Go do not natively support externref types for function parameters or return values, making it challenging to pass complex data between Wasm modules and their host environments effectively.

> Reference type (aka externref or anyref) is an opaque reference made available to a WASM module by the host environment. Such references cannot be forged in the WASM code and can be associated with arbitrary host data, thus making them a good alternative to ad-hoc handles (e.g., numeric ones). References cannot be stored in WASM linear memory; they are confined to the stack and tables with externref elements.
>
> Rust does not support reference types natively; there is no way to produce an import / export that has externref as an argument or a return type. wasm-bindgen patches WASM if externrefs are enabled. This library strives to accomplish the same goal for generic low-level WASM ABIs (wasm-bindgen is specialized for browser hosts).

> **externref use cases** Since externrefs are completely opaque from the module perspective, the only way to use them is to send an externref back to the host as an argument of an imported function. (Depending on the function semantics, the call may or may not consume the externref and may or may not modify the underlying data; this is not reflected by the WASM function signature.) An externref cannot be dereferenced by the module, thus, the module cannot directly access or modify the data behind the reference. Indeed, the module cannot even be sure which kind of data is being referenced. - https://docs.rs/externref/latest/externref/

Even if externref were fully implemented in future specs for Go (or Rust) such that it could be used as an argument or return type across modules (still not ideal for type-checking as it is not the underlying type), this would limit what can be inter-module-referenced to that which can be held in memory. The Gno Virtual Machine (GnoVM) allows for inter-user-package (inter-realm) references across the entire persisted disk store space, and does not require any additional language syntax such as with the `externref` keyword, and supports the normal course of type-checking already familiar to Go developers.

3.2 Interrealm Programming

Gno extends Go with explicit multi-user realm awareness. The key concepts:

- **Realm-context**: determines `runtime.CurrentRealm()` and `runtime.PreviousRealm()`, controlling identity and agency.
- **Realm-storage-context**: determines where new and modified objects are persisted during finalization.
- **Crossing-functions**: declared with `func fn(cur realm, ...)` and called with `fn(cross, ...)` to explicitly change both contexts.
- **Readonly taint**: values accessed via dot-selectors or index-expressions on external realm objects are tainted read-only, preventing direct mutation even by logic declared in the same realm. This protects against a class of exploits where a realm is tricked into mutating its own objects.
- **Realm-transaction finalization**: at realm boundaries, new objects are assigned IDs and persisted, ref-count-zero objects are deleted, and Merkle hashes are recomputed.

```go
// realm /r/bob/bob
package bob

import "gno.land/r/alice/alice" // import external realm package
import "gno.land/p/bob/types"   // import external library package

func Register(cur realm, name string) {
    prof := types.UserProfile{Name: name}
    alice.SetObject(cross, prof) // explicit realm crossing
}
```

Non-crossing methods on receivers residing in external realms implicitly change only the realm-storage-context (not the realm-context), allowing methods to behave identically whether declared in a p package or realm package. Panics that cross realm boundaries abort the program; `revive(fn)` provides software transactional memory semantics for testing (and eventually production use). Future builtins include `attach()` for binding

unreal objects to a realm-storage-context, and `safely(cb)` for clearing realm-contexts to prevent yielding agency to callees.

For the full specification including the object model (real/unreal/escaped objects), read-only taint rules, crossing semantics, and design goals, see the Interrealm Specification.

4 Gno.land Blockchain

Tendermint solved proof-of-stake by innovating upon classical Byzantine fault-tolerant consensus algorithms published by Dwork, Lynch, and Stockmeyer in 1988 (originally funded by Darpa for missile defense systems) for blockchains. It paved the way for the Cosmos Hub, the first proof-of-stake IBC hub, and Cosmos the internet of blockchains. Also of note, when Binance first launched they used the CosmosSDK and Tendermint.

Gno.land builds upon Tendermint2 and aims to shift the paradigm of programming languages in general: Gno is the first *multi-user* programming language, making it a superior smart contracting language as compared to any existing solution. Thus **Gno.land is the first multi-user language-based operating system**. Its ultimate goal is to be the world's open knowledge base for the next millennium.

4.1 Use Case: Open Programmable Knowledge Base

Go's simple embedded struct-centric design and the Gno VM's automatic transactional persistence makes Gno.land not only great for decentralized financial applications but also makes it uniquely well suited and designed for permissionless innovation of information-based applications such as social communication and coordination systems, or the next Wikipedia or programmable knowledge-base systems. The latter will be explored here.

Each of the thought statements in the introduction can be represented as a simple Go string, but as in Tractatus we want to allow each of these thought- statements to be supported by any number of supporting thought statements, so we need a struct declaration.

```
type Thought struct {
    Statement    string
    Dependencies []*Thought
}
```

The above allows for a simple tree structure, but it would be better to annotate each child node (thought statement) with the type of relation to the parent node– for example whether a child node represents an example, a caveat, a corollary, or supporting evidence and so on.

```
// Option "Denormalized Thought"
type Thought struct {
    Statement   string
    Examples    []*Thought
    Caveats     []*Thought
    Corollaries []*Thought
    Support     []*Thought
}
```

Better than a denormalized structure is one where the type of thought statement association can be arbitrary or fixed depending on the application.

```
// Option "Normalized Thought"
type Thought struct {
    Text         string
    TypedSupport []*Thought
}

type ThoughtType string // examples, caveats, corollaries, support
```

```go

type TypedThought struct {
    Type    ThoughtType
    Thought *Thought
}
```

Note on the usage of `[]\*Thought` slices: in the current implementation of the GnoVM each slice can only be used by first loading the entire underlying base array. This may be optimized in the future, however for holding large sets of elements the programmer should instead use a tree-structure such as the avl.Tree (or an iavl.Tree).

Now arises the question of whether counter-arguments should also be referenced as a child node to the original thought parent node. If we include counter-arguments in the graph of `*Thought` objects itself there is the issue of permissioning who can add counter-arguments to the graph. With the examples above and with no method declarations a `*Thought` belonging to one user cannot be modified by a third party even though the fields of a `Thought` struct is exposed due to Gno (runtime) interrealm rules that taint third party reads via direct dot-selectors & index-expressions with a readonly-taint that persists even with (direct selector) access of sub-fields.

The `*Thought` object can however be modified by another user by calling a declared method. We can extend the `Thought` struct with additional fields for authorization or ownership and implement a method such as follows:

```go
type Thought struct {
    Owner        account
    Statement    string
    TypedSupport []*Thought
}

func (th *Thought) AddCounterArgument(cth *Thought) {
    caller := runtime.CurrentRealm().Address
    if th.Owner != caller {
        panic("unauthorized")
    }
    th.TypedSupport = append(th.TypedSupport,
        TypedThought{Type: "counter", Thought: cth})
}
```

This works but not well–it only works if the owner of the parent node wants the counter-argument to be registered. Even if counter-arguments were not registered as an assocation on chain, it is still possible for any Gno.land state indexer to separately index the reverse association of reference to the original `*Thought` when it finds a counter-argument `*Thought` that references in its struct field the original as a counter-argument. This reliance on an external indexer shifts trust from the blockchain itself to the indexer so is not always ideal.

Gno is intended for permissionless iteration and improvement so we will discuss another way (among many) to manage associations of competing thought statements; the pair-wise association among competing thought statements can be registered in another (neutral) external realm that allows the registration only at least one of the two thought statements identify the other as a counter-argument. In this case it is not necessary for a `*Thought` object to be associated with any owner explicitly (via the `.Owner` field). Note however that given the Gno inter-realm specification to make a `*Thought` object truly

immutable even for the owner of the realm in which it resides it must not expose any mutator functions, or it should have at least a `readonly bool` field.

We can also add discussion board objects for each thought statement.

```go
// Board defines a type for boards.
type Board struct {
    // ID is the unique identifier of the board.
    ID ID
    // Name is the current name of the board.
    Name string
    // Aliases contains a list of alternative names for the board.
    Aliases []string
    // Readonly indicates that the board is readonly.
    Readonly bool
    // Threads contains all board threads.
    Threads PostStorage
    // ThreadsSequence generates sequential ID for new threads.
    ThreadsSequence IdentifierGenerator
    // Permissions enables support for permissioned boards.
    // This type of boards allows managing members with roles and permissions.
    // It also enables the implementation of permissioned execution of board related features.
    Permissions Permissions
    // Creator is the account address that created the board.
    Creator address
    // Meta allows storing board metadata.
    Meta any
    // CreatedAt is the board's creation time.
    CreatedAt time.Time
    // UpdatedAt is the board's update time.
    UpdatedAt time.Time
}

// New creates a new basic non permissioned board.
func New(id ID) *Board {
    return &Board{
        ID:              id,
        Threads:         NewPostStorage(),
        ThreadsSequence: NewIdentifierGenerator(),
        CreatedAt:       time.Now(),
    }
}
```

While it is certainly possible to embed a `*Board` as a field of each `*Thought`, the current implementation of `*Board` is only safe from a moderation perspective when it is permissioned; and so a board tightly coupled to a `*Thought` may not be ideal depending on the use-case. Instead we can map an external realm persisted index of `*Thought` to `*Board` associations similarly to how counter-thoughts are associated as mentioned before. In both cases we probably want to add to the `Thought` struct a globally unique ID like how `Board` has. *In the future we may provide a standard function to get a unique identifier for every pointer object but this has not yet been decided yet.*

Finally, consider for example the numbered sequence of verses of a book of the bible, or the deep tree of statements in Wittgenstein's Tractatus. In order to facilitate the efficient forking of such large lists or graphs of objects it is necessary to avoid the usage of slices. Even the avl.Tree (as provided in the Gno monorepo under the examples directory) is not sufficient as it is a mutable tree. However a fork of the avl.Tree into an immutable tree (or likewise a port of the iavl tree in the tm2 Tendermint2 directory) can be used with

some improvement to allow for splicing in new elements and deleting existing elements from the original tree.

So far I have illustrated a way for multiple users to construct their thought statement graphs independently while also allowing for counter-arguments to be registered/associated permissionlessly. More design and exploration is needed to create a fully functional permissionlessly extensible thought statement graph system; and in the primordial soup of Gno ecosystem eventually one or more designs will become dominant in usage by evolution. The reader is encouraged to explore the above template and measure success by references and by forks. See also Use Case: 95 Facts.

4.2 Use Case: Home Computing Chains

The Gno VM is not just useful in the context of public decentralized blockchains. It is also useful for home computing. Take for example Email which despite all attempts to replace it still persists in our lives today as flawed, complex, and outdated as it is. While a realm that stores mail on the blockchain is not useful unless the data is encrypted *(and even if it were encrypted it is not a good idea nor encouraged to store encrypted data on gno.land as encryption keys may eventually get acquired by hackers and leaked and even persisted on the blockchain too)*, the Gno VM can run anywhere, even on your private server hosted at home. In fact, this is what we should do given the prevalence of surveillance technology such as Google's Gmail which uses AI to sort your mail and analyze for targeted advertising.

Imagine a black box local GnoVM you run at home. You can have the `/r/home/email` realm store your emails at home on your own home server. The same blockchain node logic can run on its own as a single-validator *home chain* which naturally supports backups as secondary full nodes, or you can even make your home chain byzantine fault-tolerant for better uptime.

1. Install in your home GnoVM chain a service plugin: /s/email/indexer, not /p/* nor /r/* but /s/* for off-chain service applications. *(this prefix is not supported in gno.land but may be in the future).*

- /s/email/indexer reads state upon init, but also registers as a listener for notifications from /r/emails.
- When a new email comes in, /r/emails via listeners calls `/s/email/indexer.AddEmail()`.
- /s/email/indexer also imports /d/email/indexer which is an off-chain daemon component. Here /d/* represents a hypothetical prefix for Gno code to be run off-chain with arbitrary Go native functions available for import that would otherwise not be possible on gno.land (since a blockchain can only support deterministic logic).
- /d/email/indexer can only access /s/email/indexer by a Gno firewall system declared with Gno package paths, types, and function/method names.
- /s/email/indexer can import any /r/* or /p/* but not any /s/* (like Chrome extensions) and its own /d/email/indexer, unless otherwise restricted by the Gno firewall system.

- Your mobile device registers an account with your local GnoVM home chain. This phone account is restricted to only access /s/email/indexer.
- Your phone makes a request to /s/email/indexer. It then asks /d/email/indexer which in turn queries the local index and responds via /s/email/indexer.

Here are some benefits of GnoVM home computing: * Gno.land can be leveraged to ensure that all software is properly audited. * Software is expected to become finished and immutable. * All software benefits from Go/Gno's type-safety and memory-safety. * A unified IPC system drastically reduces surface area for penetration. * Plugin services and daemons such as the aforementioned email indexer can be containerized and restricted from unauthorized access. * Fine-grained security/firewall rules at the function invocation level. * Byzantine fault-tolerance comes out of the box for zero downtime. * Inversion of control with public key cryptography for everything: no more password management. * Resilience against infrastructure failures: home chains continue operating during internet outages, power grid disruptions, or state-level network shutdowns. Combined with mesh networking and local power (solar, battery), GnoVM home chains can form the backbone of community-level communication and coordination when centralized infrastructure fails.

4.3 Use Case: Supply Chain Transparency and Food Sovereignty

The fragility of global supply chains has become impossible to ignore. From semiconductor shortages to food supply disruptions, dependency on opaque, centralized, and geographically concentrated supply chains puts populations at the mercy of forces they cannot see, verify, or influence. The geopolitical tension surrounding Taiwan and TSMC, the concentration of pharmaceutical manufacturing in China and India, and the consolidation of food production under a handful of multinational corporations all represent single points of failure with civilizational consequences.

Gno contracts can implement transparent, accountable supply chain tracking where every step from origin to consumer is recorded with cryptographic accountability. Unlike centralized supply chain solutions (which are only as trustworthy as the company operating them), Gno-based supply chain modules are open source, auditable, and composable. A food provenance realm can import a certification realm which imports a reputation realm–all with type-safe interoperability and no middleware.

For food sovereignty specifically, local communities can use Gno.land to coordinate direct farmer-to-consumer networks, manage community-supported agriculture (CSA) memberships, and maintain transparent records of growing practices–all without depending on any centralized platform that could deplatform them or change terms of service. When combined with home computing chains, local food networks can operate entirely independently of the global internet if necessary.

4.4 Other Use Cases

Gno.land can be used to host any other smart-contract application supported by Ethereum written in Solidity, such as Defi applications, name-resolution systems, DAOs and governance applications, etc.

You can explore the various dapps including sample implementations of ERC equivalents in the examples directory. *Note that these prototypes have not yet been audited unless*

otherwise specified!

There are also many good use-cases for integrating the GnoVM in client or server side programming where deterministic gas-limited (for CPU, disk, memory) execution of logic is desired. In particular Gno and the Gno VM can be ideal for scripting in the gaming context for multi-player games. There are many games like Minecraft (with JavaScript) or Second Life (with LSL) that offer scripting in the game, but they are usually not both deterministic and gas-counted; but these are the properties you may need in a multi-player game if you also want to provide fault-tolerant, accountability, and replayability guarantees (with or without decentralization or replication).

The main feature of Gno is its multi-user context awareness. Where-ever you want user provided logic directly interacting with those of another user Gno solutions will be more succinct, elegant, and easy to develop and maintain.

4.5 Gno.land Constitution

See [./CONSTITUTION.md] for the Gno.land Constitution (draft) and details of genesis, tokenomics, governance, and more.

Separation of Church and State Madison separated church and state in the US Constitution albeit there is a hint of the Christian spirit by the way in which the constitution was signed: "... in the Year of the Lord...". All the founders were Christian including Jefferson and Madison, and in particular the primary author of the US Constitution James Madison explicitly separated church from the constitution so as to help promote the teachings of Jesus as evidenced in his other writings. Likewise Gno.land besides this whitepaper is independent of any religion by its constitution, which should only refer to this whitepaper sparingly.

Gno.land will launch with a minimal (living) constitution written and maintained in English, but also ultimately be supplemented by the completed GnoVM code and Tendermint2 and Gno.land implementation. Future implementations of the GnoVM and Gno.land should adhere to the completed software mentioned above.

Gno.land should not censor speech, even if the speech is wrong. However, it should ban all porn and try to limit external links to porn sites as porn is not speech and is dangerous to civilization. Whether hate-speech is tolerated should be determined by each realm but also by the living Gno.land constitution and by GovDAO vote to amend the constitution and laws of Gno.land.

4.6 GnoWeb Browser

GnoWeb is the server software for Gno.land, a browser within a browser for viewing realm data.

Instead of requiring realm applications to return HTML, the convention is to implement a Render() function that returns Markdown. This is to allow the transition away from the bloated HTML standards and browser software and realm data to be browsed even from the console. Note that we don't need HTML XML elements to denote objects: everything in Gno is already an object. This makes Gno.land more like the original World Wide Web that conforms to the *Document Object* model (DOM).

GnoWeb does support some custom XML elements for improved layout and functionality, such as column layout and form submissions that integrate with browser extensions for transaction signing.

Note that GnoWeb is not yet a general blockchain explorer (e.g. for transactions) nor a general purpose Gno.land state explorer. A Gno.land specific blockchain explorer already exists. GnoWeb can only render markdown returned from Render() functions, and a general purpose state explorer is still desired.

Realm code is not precluded from returning HTML or even JSON for custom browser applications. In the near future the Gno.land node software will support returning JSON encodings of Gno objects. Thus future alternative browser applications may provide more interactive rich user experiences for viewing and mutating Gno.land state without any Markdown intermediary representation; and perhaps leveraging AI for intelligent layout and styling for rich interactivity.

5 Future Work

- **Access control systems.** A composable library of membership, reputation, and moderation primitives that realms can import and combine without any single mandated identity system. Components include web-of-trust scoring, deposit-based access, viral invitation trees, hierarchical accountability chains, and oracle-backed attestations. The goal is a free market of access control policies rather than a universal gatekeeping layer. GnoAuth is a first step: an OAuth identity oracle that binds Web2.0 accounts (e.g. GitHub) to Gno addresses with signed attestations and sybil-resistance metadata.
- **Persistent garbage collector.** The current implementation persists objects with reference counting and deletes zero-ref-count objects at realm-transaction finalization, but does not yet collect reference cycles. A full persistent cycle-detecting garbage collector will reclaim storage for cyclic structures that become unreachable. A transaction-scoped cyclic GC may also be implemented as a lighter alternative for common cases.
- **Name registry for realm and package paths.** A decentralized naming system for mapping human-readable names to realm and package paths, enabling stable references even when packages are re-deployed or versioned. This would function similarly to DNS but for the Gno package namespace.
- **Realm upgrading.** A mechanism for upgrading mutable realm code while preserving existing state and object references. This requires careful design to maintain type safety across upgrades and to ensure that dependent realms are not broken by upstream changes. Migration functions and versioned type schemas are under consideration.
- **Realm data browser.** A general-purpose state explorer for inspecting any realm's persisted object graph, not limited to Render() output. This would allow developers and users to browse the full object tree, inspect individual objects, view ownership and reference graphs, and trace cross-realm references.
- **Deterministic concurrency.** Extending Gno with deterministic concurrency primitives that preserve the reproducibility required for consensus while enabling parallel execution of independent realm transactions. This could unlock significant throughput improvements without sacrificing the determinism guarantee.
- **Joeson parser.** A general-purpose parser library implemented in Gno for on-chain parsing of structured data formats, domain-specific languages, and protocol messages. This enables realms to define and interpret custom grammars without relying on off-chain tooling.
- **Gno2.** A future major revision of the Gno language incorporating lessons learned from production use. Potential additions include a `readonly` modifier for function return types, native syntax for `cross` and `cur realm` (replacing the current overloaded function argument convention), and further language-level support for multi-user patterns that emerge from ecosystem usage.
- **Open hardware.** Memristor-based memory systems eliminate the distinction between volatile RAM and persistent storage, which aligns naturally with the

GnoVM's automatic persistence model. Purpose-built hardware could further simplify the VM and dramatically improve performance. Open hardware designs for validator nodes would also strengthen decentralization by reducing dependence on proprietary chip manufacturers.